Dal codice al container

Docker e Azure Container Registry in pratica

Nicola Paro

SPONSOR

UNIPORDENONE
moltiplica il valore della formazione

TD SYNNEX

An unhandled exception occurred while processing the request.

Win32Exception: An error occurred trying to start process 'ffmpeg' with working directory 'C:\Users\nicol\source\GlobalAzure26\YouTubeToMp3'. The system cannot find the file specified.

System.Diagnostics.Process.StartWithCreateProcess(ProcessStartInfo startInfo)

Stack Query Cookies Headers Routing

Win32Exception: An error occurred trying to start process 'ffmpeg' with working directory 'C:\Users\nicol\source\GlobalAzure26\YouTubeToMp3'. The system cannot find the file specified.

System.Diagnostics.Process.StartWithCreateProcess(ProcessStartInfo startInfo)

Program+<>c+<<<Main>$>b__0_0>d.MoveNext() in **Program.cs**

```
57.         ffmpeg.Start();
```

Microsoft.AspNetCore.Http.RequestDelegateFactory.ExecuteTaskResult<T>(Task<T> task, HttpContext httpContext)

Microsoft.AspNetCore.Diagnostics.DeveloperExceptionPageMiddlewareImpl.Invoke(HttpContext context)

Show raw exception details

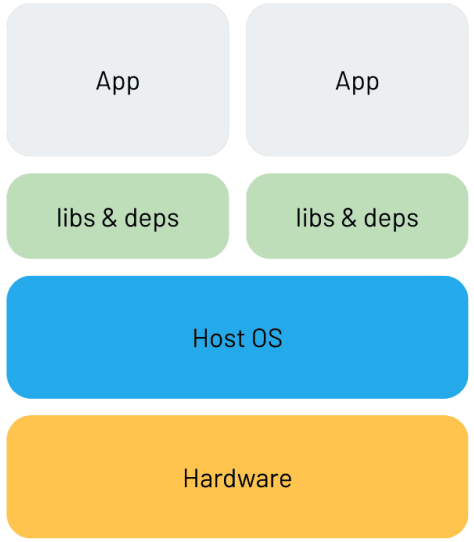
«Non funziona niente»

Sul mio computer girava tutto perfettamente

Allora portiamo il tuo computer dal cliente

Perfetto, così mi compri un computer nuovo

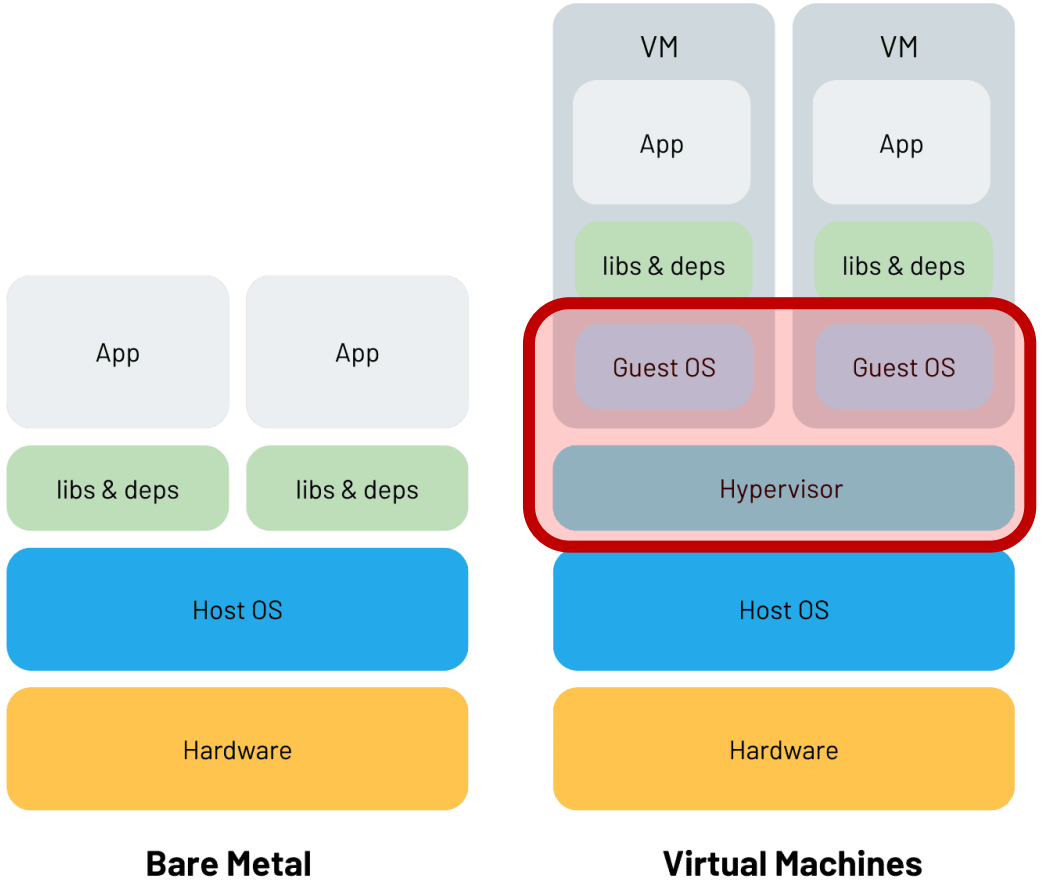
Più modi di deployare



Bare Metal

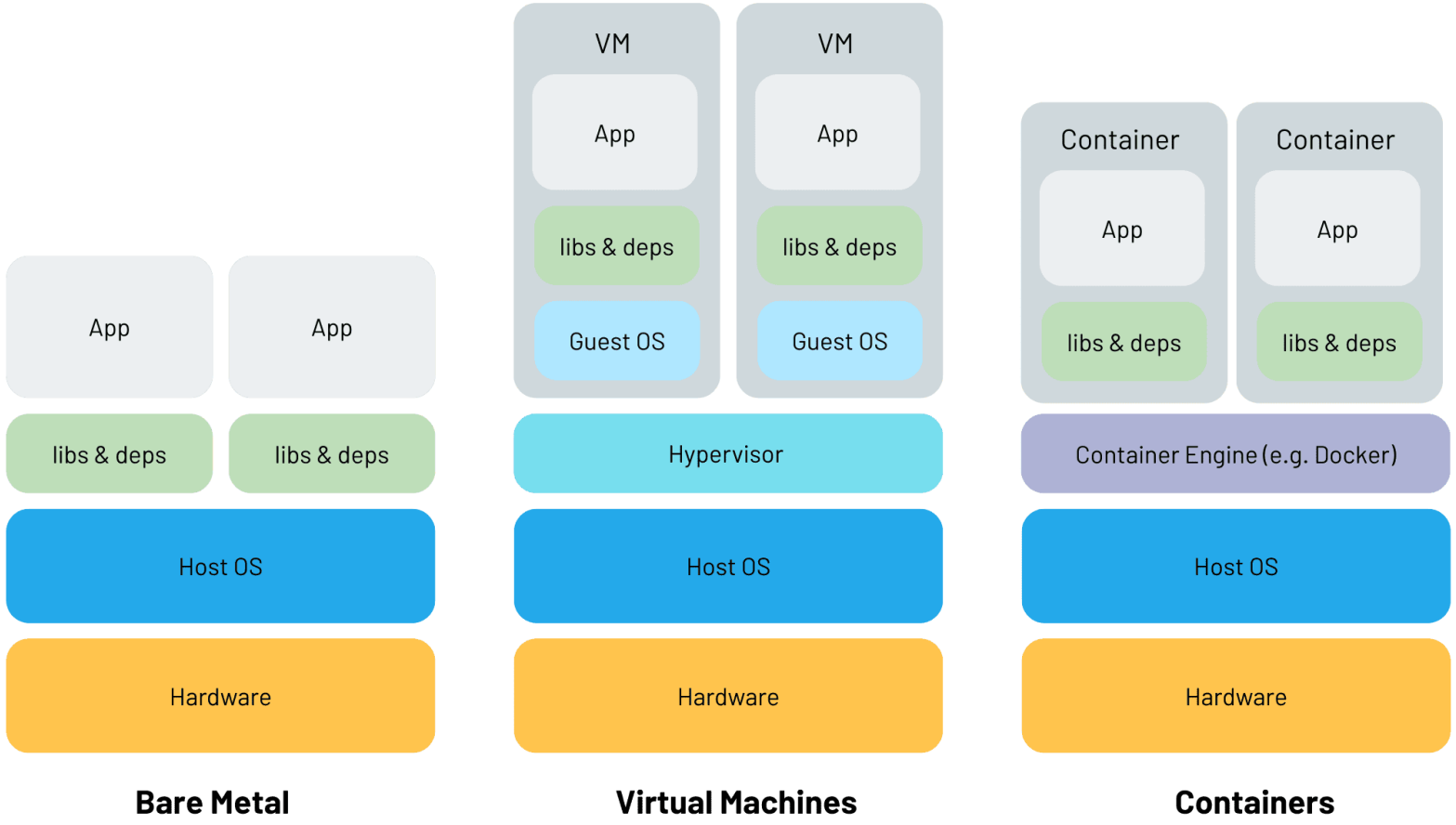
- Variabili di ambiente mancanti
- Runtime non perfettamente allineati con quelli di dev
- OS non esattamente lo stesso
- DLL Hell
- «Ho messo l'antivirus e ora il tuo software non funziona più»

Più modi di deployare



- Tempi di avvio
- Spazio Occupato
- CPU e RAM

Più modi di deployare



Containers

Cos’è un container?

Un container è un’unità software leggera che include

- Codice dell’applicazione
- Dipendenze
- Librerie
- Configurazioni

Come funziona un container?

- I container condividono lo stesso kernel del sistema operativo
- Non simulano hardware (a differenza delle VM)

Linux Containers (LXC)

Come funziona

LXC è una **virtualizzazione a livello di sistema operativo**.

Usa funzionalità del kernel Linux:

- **namespaces** → isolamento (processi, rete, filesystem)
- **cgroups** → gestione risorse (CPU, RAM, I/O)

Tutti i container condividono **lo stesso kernel**

Ogni container è simile a **una mini macchina virtuale Linux**

Caratteristiche

- Container **multi-processo**
- Puoi fare SSH dentro il container
- Ambiente simile a un sistema Linux reale
- Più orientato a **sysadmin / infrastruttura**

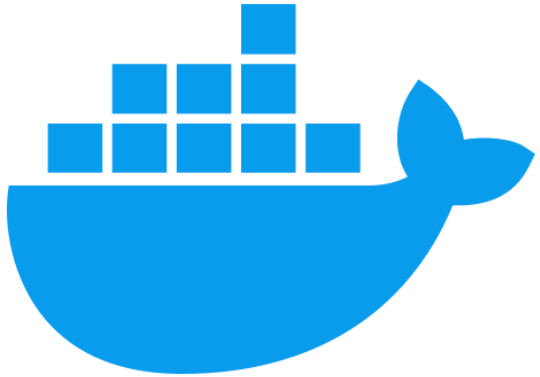
Docker

Docker è **una piattaforma completa di containerizzazione**.

1. **Immagini**
   - Template immutabili con app + dipendenze
2. **Container**
   - Istanza runtime di un'immagine
3. **Docker Engine**
   - Runtime che gestisce tutto
4. **Dockerfile**
   - Script per creare immagini automaticamente

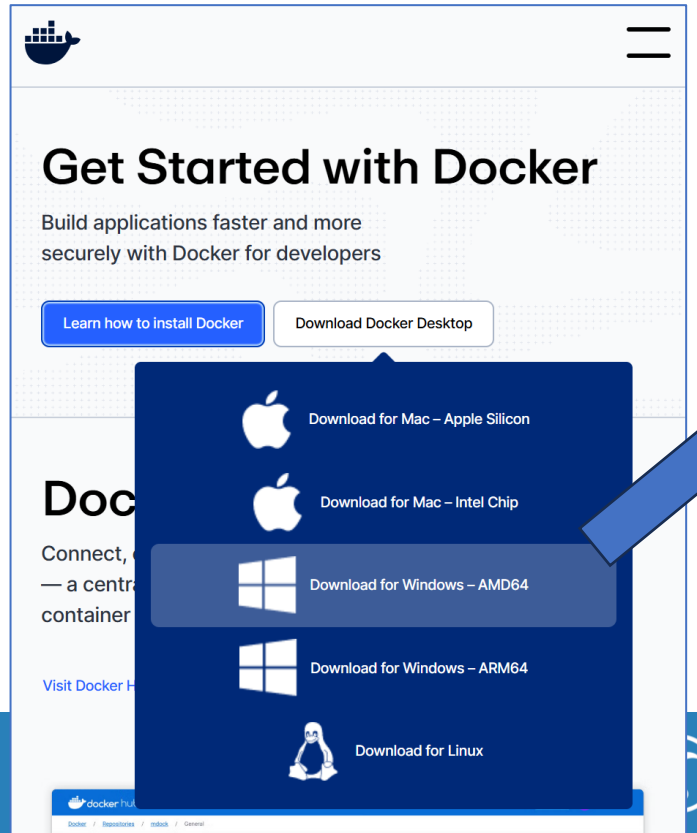
Caratteristiche

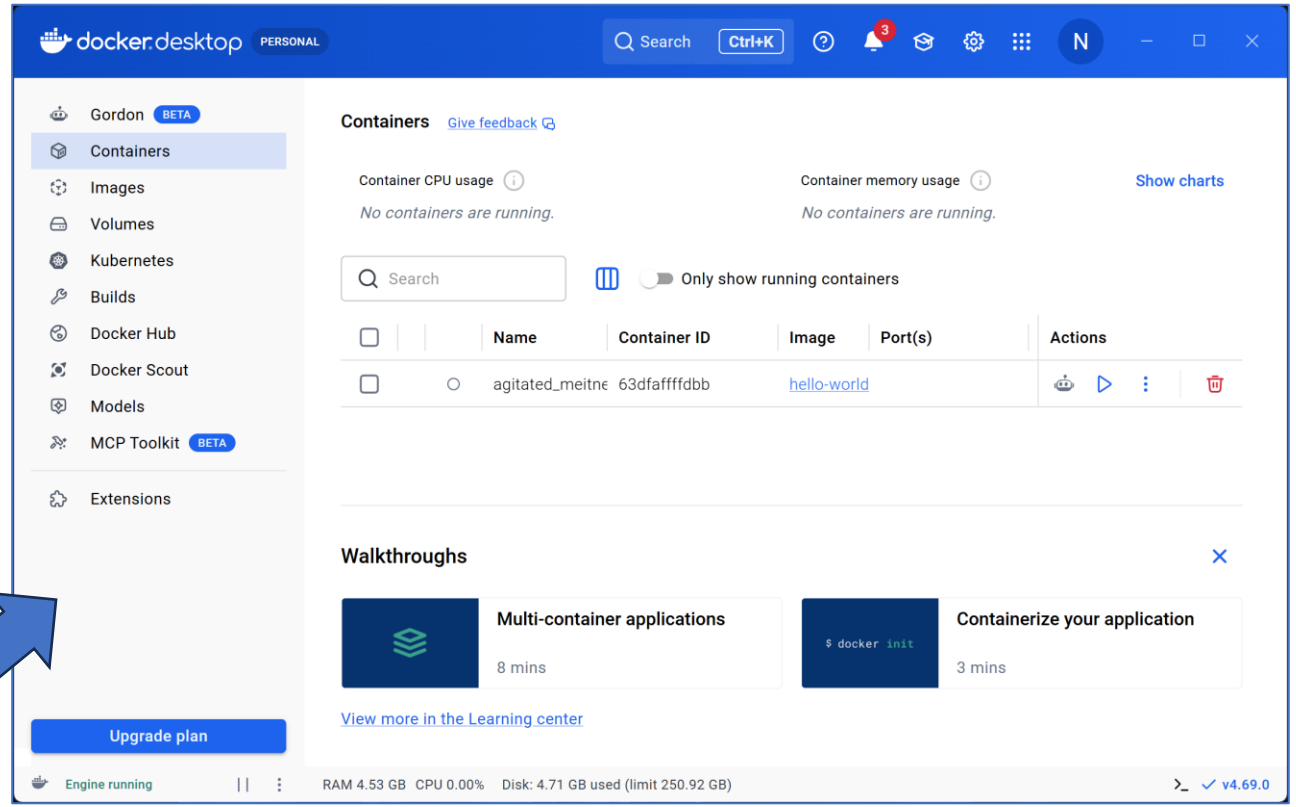
- Un container Docker = una singola applicazione
- Non simula un OS completo
- Ambiente minimale



Docker – Getting Started

https://www.docker.com/





Docker Desktop – Pricing

Docker Personal	Docker Pro	MOST POPULAR Docker Team	Docker Business
$0	**$11** per user/month	**$16** per user/month	**$24** per user/month
For individual developers who need the essential tools to build and deploy containers.	For individual professionals who require more advanced features and additional resources.	For small teams that need collaborative tools to make working together more efficient.	For enterprises desiring robust security, control, and compliance features.
Get started	Buy now	Buy now	Contact sales Buy now

Se l'azienda ha **250+ dipendenti** oppure fattura oltre **10 milioni** di dollari l'anno, è obbligatorio avere una licenza a **pagamento**.

È possibile avere Docker Gratis?

- **Docker Engine** installato direttamente **su Linux** (tramite apt, yum, ecc.) è rilasciato sotto **licenza Apache 2.0** ed è libero per qualsiasi uso, incluso quello commerciale, senza restrizioni di fatturato o numero di dipendenti.
  - Questo include anche WSL, per cui è possibile utilizzare Docker Engine anche su macchine Windows
- Il Docker Engine ottenuto tramite Docker Desktop (anche su Linux) richiede una sottoscrizione a pagamento per le grandi imprese che superano i 250 dipendenti o i 10 milioni di dollari di fatturato annuo.

Docker Gratis?

```
sudo apt update && sudo apt upgrade
```

```
sudo apt-get install ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo tee
/etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc]
https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME")
stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-
compose-plugin
```

```
sudo docker run hello-world
```

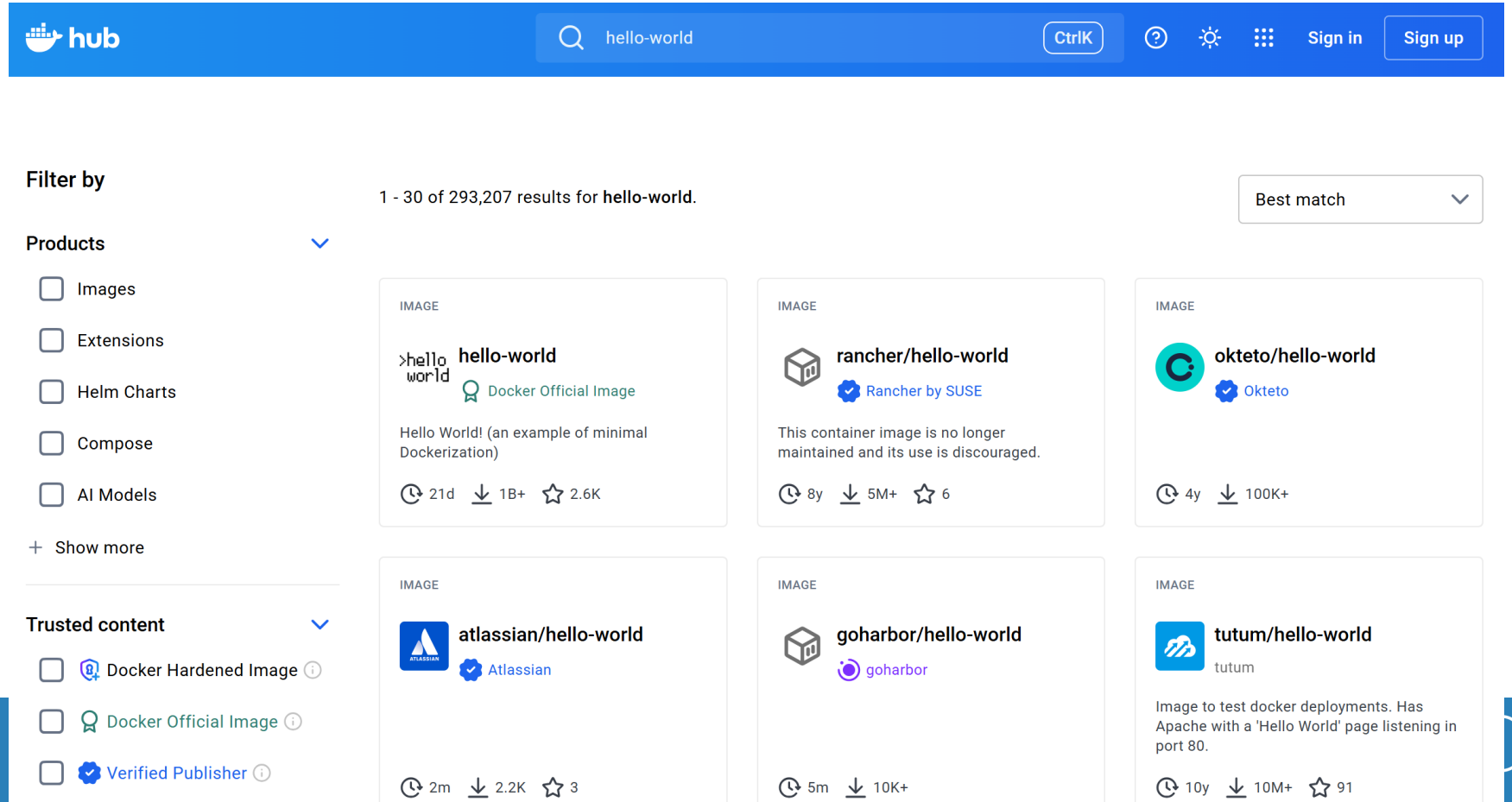
docker run hello-world

È il comando principale di Docker

1. Crea ed avvia un container partendo da un’immagine.
2. Prende un’immagine (se non è presente la scarica)
3. Crea un container
4. Esegue un comando dentro il container
5. Lascia in esecuzione / lo ferma (a seconda delle opzioni) .

Da dove si scaricano le immagini?

Da Docker Hub, che è il registro pubblico ufficiale di Docker.



Esiste solo Docker Hub?

Se nel nome dell'immagine si indica un registry, Docker scarica da lì:

docker run google/cloud-sdk → Docker Hub

docker run quay.io/coreos/etcd → Quay.io

docker run public.ecr.aws/nginx/nginx → AWS ECR

docker run mylocalregistry.local:5000/app → Registry Privato

Docker per le immagini private?

Se nel nome dell'immagine si indica un registry, Docker scarica da lì:

```
docker login registry.local
docker run registry.local/mia-immagine
```

Ma il cliente non può accedere al mio registry locale…

Azure Container Registry

È il Container Registry privato gestito da Microsoft Azure.

Serve per archiviare, versionare e distribuire immagini container (Docker/OCI) all'interno del tuo ambiente Azure, invece di usare registry pubblici come Docker Hub.

È organizzato in repository + tag.

Azure Container Registry – Tier & Pricing

	Basic	Standard	Premium
Price per day	**€0.145**	**€0.579**	**€1.447**
Included storage (GB)	10	100	500 Premium offers enhanced throughput for docker pulls across multiple, concurrent nodes
Total web hooks	2	10	500
Geo Replication	Not Supported	Not Supported	Supported **€1.447** per replicated region
Connected registry	Not Supported	Not Supported	Supported **€8.970** per connected registry per month

Azure Container Registry - Features

🔐 Access Control

- Integrazione nativa con Microsoft Entra ID.
- Controllo accessi tramite RBAC.
- Service principal / managed identity / token scoped
- Private Endpoint / Private Link
- Possibilità di disabilitare l'admin user

🛡️ Security & Compliance

- Image scanning tramite Microsoft Defender for Containers (vulnerability assessment).
- Content Trust / image signing per garantire l'integrità delle immagini.
- Quarantine e policy di sicurezza (feature avanzate, SKU Premium).
- Supporto a encryption con customer-managed keys (CMK).

⚙️ ACR Tasks

- ACR Tasks per build automatizzate nel cloud:
- build on-demand (az acr build)
- trigger su commit Git
- rebuild automatico quando cambia l'immagine base
- Supporto a build multi-step e pipeline container-native.
- Integrazione diretta con Azure DevOps, GitHub, Jenkins.

Come creo un’immagine Docker?

Creare un’immagine Docker significa impacchettare un’applicazione e le sue dipendenze in un formato standard che può essere eseguito ovunque.

Dockerfile

- È la ricetta che dice a Docker come costruire un’immagine.
  - una descrizione dichiarativa di come nasce un ambiente
  - uno standard ripetibile e versionabile

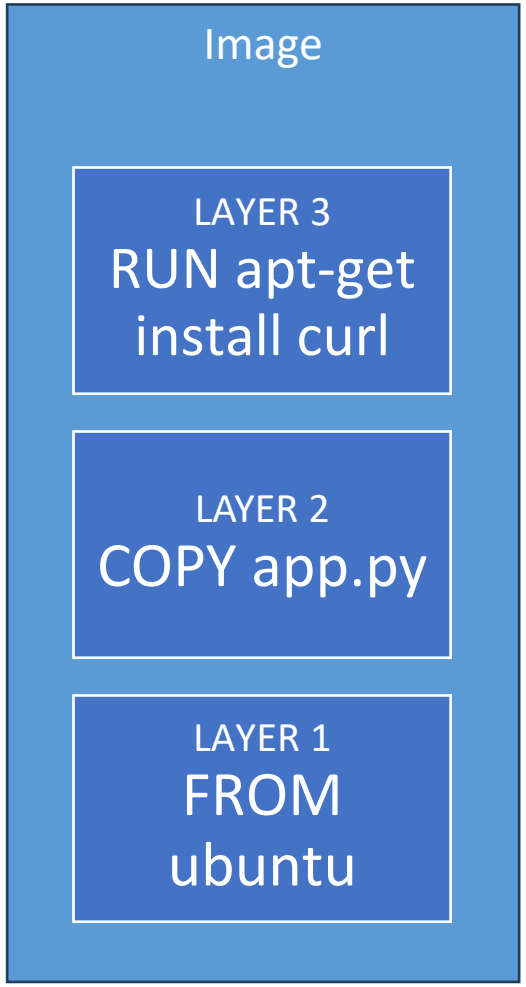
- Durante la build, docker legge il Dockerfile dall’alto verso il basso, esegue ogni istruzione in sequenza e crea una immagine finale immutabile

Dockerfile – Istruzioni

Istruzione	Crea un layer?	Quando si usa	Descrizione
FROM	✅	Sempre (quasi)	Immagine base da cui partire. Può comparire più volte (multi-stage build).
RUN	✅	Build time	Esegue comandi durante la build (installazioni, setup). Ogni RUN crea un layer.
COPY	✅	Build time	Copia file/directory dal filesystem locale all'immagine.
ADD	✅	Build time	Come COPY, ma può estrarre archivi e accettare URL (uso limitato).
WORKDIR	❌	Build + runtime	Imposta la directory di lavoro. Se non esiste, viene creata.
CMD	❌	Runtime	Comando di default all'avvio del container. Sovrascrivibile.
ENTRYPOINT	❌	Runtime	Comando fisso all'avvio del container. Usato per container "eseguibili".
ENV	❌	Build + runtime	Imposta variabili d'ambiente persistenti nell'immagine.
ARG	❌	Build time	Variabili disponibili solo durante la build (non a runtime).
EXPOSE	❌	Metadata	Documenta la porta usata dall'app (non la pubblica).
USER	❌	Build + runtime	Imposta l'utente con cui vengono eseguiti i comandi/container.
VOLUME	❌	Runtime	Definisce un mount point per volumi persistenti.
LABEL	❌	Metadata	Aggiunge metadati all'immagine (versione, maintainer, ecc.).
SHELL	❌	Build time	Cambia la shell usata dai comandi RUN.
STOPSIGNAL	❌	Runtime	Specifica il segnale di stop per il container.
ONBUILD	❌	Build (immagini figlie)	Definisce trigger eseguiti quando l'immagine è usata come base.
HEALTHCHECK	❌	Runtime	Definisce un comando per verificare lo stato di salute del container.

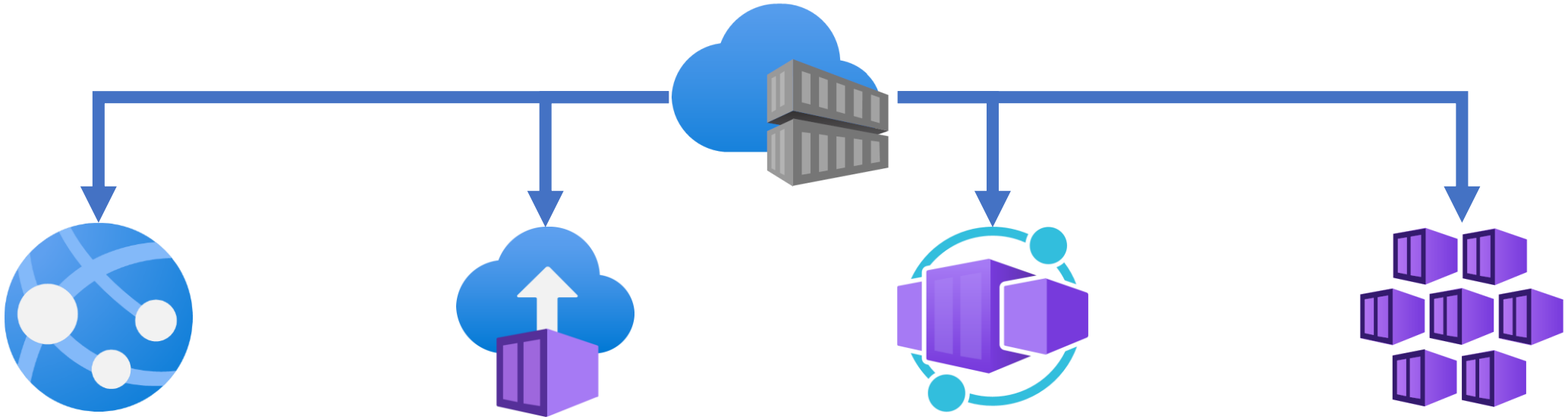
Layers?

- Un’immagine docker è costituita da Layers
  - Un layer è una modifica immutabile del filesystem
  - Ogni layer rappresenta: un’aggiuntauna rimozioneo una modifica ai file
- L’immagine finale è la somma ordinata dei layer
- Perché servono i layers?
  1. Caching (velocità)
  2. Riutilizzo tra immagini
  3. Immutabilità



DEMO

Ora che abbiamo una Container Registry…



Web Apps

Container Instances

Container Apps

Azure Kubernetes Services

Da zero al cloud con .NET Aspire: crea e pubblica su Azure Container Apps
Marco Bortolin

A quick introduction to AKS
Alessandro Melchiori

Grazie!

Nicola Paro

Solution Architect

Microsoft® MVP Most Valuable Professional

linkedin.com/in/nicolaparo